08-상태-동기화

개요

이 문서는 Monad 블록체인의 상태 동기화(State Synchronization) 프로토콜을 분석합니다. 상태 동기화는 새로운 노드가 네트워크에 참여하거나 뒤처진 노드가 최신 상태를 따라잡을 때 사용되는 핵심 메커니즘입니다.

Monad의 상태 동기화는 다음과 같은 특징을 가집니다:

- **실행/합의 분리**: 합의 컴포넌트(monad-node)와 실행 컴포넌트(monad) 간 명확한 인터페이스
- **증분 동기화**: 전체 상태가 아닌 변경사항만 전송
- **병렬 다운로드**: 여러 prefix를 동시에 동기화
- **삭제 추적**: 계정 및 스토리지 삭제를 명시적으로 전송
- **검증 가능성**: Merkle 루트를 통한 상태 무결성 검증

주요 컴포넌트

- **Client**: 상태를 받는 노드 (동기화 대상)
- **Server**: 상태를 제공하는 노드 (동기화 소스)
- **Protocol**: 클라이언트와 서버 간 통신 프로토콜

구조

관련 파일 및 디렉토리

```
category/statesync/
```

```
├── statesync_client.h # 클라이언트 C API
├── statesync_client.cpp # 클라이언트 구현
├── statesync_client_context.hpp # 클라이언트 컨텍스트
├── statesync_client_context.cpp # 클라이언트 컨텍스트 구현
├── statesync_server.h # 서버 C API
├── statesync_server.cpp # 서버 구현
├── statesync_server_context.hpp # 서버 컨텍스트
├── statesync_server_context.cpp # 서버 컨텍스트 구현
├── statesync_protocol.hpp # 프로토콜 인터페이스
├── statesync_protocol.cpp # 프로토콜 구현
├── statesync_messages.h # 메시지 타입 정의
├── statesync_version.h # 버전 호환성
└── statesync_server_network.hpp # 네트워크 추상화
```

주요 데이터 구조

메시지 타입 (statesync_messages.h)

```
// 동기화 메시지 타입
enum monad_sync_type : uint8_t {
SYNC_TYPE_REQUEST = 0, // 동기화 요청
SYNC_TYPE_TARGET = 1, // 목표 블록 헤더
SYNC_TYPE_DONE = 2, // 동기화 완료
SYNC_TYPE_UPSERT_CODE = 3, // 코드 삽입/업데이트
```

```
SYNC_TYPE_UPSERT_ACCOUNT = 4, // 계정 삽입/업데이트

SYNC_TYPE_UPSERT_STORAGE = 5, // 스토리지 삽입/업데이트

SYNC_TYPE_UPSERT_ACCOUNT_DELETE = 6,// 계정 삭제

SYNC_TYPE_UPSERT_STORAGE_DELETE = 7,// 스토리지 삭제

SYNC_TYPE_UPSERT_HEADER = 8, // 블록 헤더

};


// 동기화 요청 메시지

struct monad_sync_request {

uint64_t prefix; // 동기화할 prefix (주소 공간 분할)

uint8_t prefix_bytes; // prefix 바이트 수 (기본 1바이트 = 256개 분할)

uint64_t target; // 목표 블록 번호

uint64_t from; // 시작 블록 번호

uint64_t until; // 종료 블록 번호

uint64_t old_target; // 이전 목표 (증분 동기화용)

};


// 동기화 완료 메시지

struct monad_sync_done {

bool success; // 성공 여부

uint64_t prefix; // 완료된 prefix

uint64_t n; // 동기화된 블록 번호
```

```
};
```

클라이언트 컨텍스트 (statesync_client_context.hpp)

```
struct monad_statesync_client_context {
monad::OnDiskMachine machine; // 디스크 I/O 머신
monad::mpt::Db db; // MPT 데이터베이스
monad::TrieDb tdb; // Trie 데이터베이스 래퍼
// 진행 상황 추적 (prefix별)
std::vector<std::pair<uint64_t, uint64_t>> progress;
// 프로토콜 버전별 핸들러
std::vector<std::unique_ptr<monad::StatesyncProtocol>> protocol;
// 블록 헤더 캐시 (최근 256개)
std::array<monad::BlockHeader, 256> hdrs;
// 목표 블록 헤더
monad::BlockHeader tgrt;
// 현재 동기화 중인 블록 번호
uint64_t current;
// 버퍼링된 스토리지 (계정 없이 스토리지만 먼저 도착한 경우)
Map<monad::Address, StorageDeltas> buffered;
// 이미 본 코드 해시 (중복 방지)
ankerl::unordered_dense::segmented_set<monad::bytes32_t> seen_code;
// 코드 저장소
Map<monad::bytes32_t, monad::byte_string> code;
```

```
// 상태 변경사항 (계정 + 스토리지)
Map<monad::Address, std::optional<StateDelta>> deltas;
// upsert 카운터 (주기적 커밋용)
uint64_t n_upserts;
// 합의 컴포넌트 콜백
void (*statesync_send_request)(
struct monad_statesync_client *, struct  monad_sync_request);
// 변경사항을 데이터베이스에 커밋
void  commit();
};
```

서버 컨텍스트 (statesync_server_context.hpp)

```
struct monad_statesync_server_context : public monad::Db {
monad::TrieDb &rw; // 읽기/쓰기 데이터베이스
monad::mpt::Db *ro; // 읽기 전용 데이터베이스
// 제안된 삭제 (아직 finalize되지 않음)
std::deque<monad::ProposedDeletions> proposals;
// Finalize된 삭제 (동기화 시 전송)
monad::FinalizedDeletions deletions;
};

// 삭제 추적 구조
```

```
struct Deletion {
Address address; // 삭제된 주소
std::optional<bytes32_t> key; // 스토리지 키 (nullopt면 계정 삭제)
};


// Finalize된 삭제 버퍼 (최대 43,200개 블록, 2백만 개 삭제)
class FinalizedDeletions {
uint64_t start_block_number_;
uint64_t end_block_number_;
std::array<FinalizedDeletionsEntry, MAX_ENTRIES> entries_;
std::array<Deletion, MAX_DELETIONS> deletions_;
size_t free_start_;
size_t free_end_;
public:
// 특정 블록의 삭제를 순회
bool for_each(uint64_t block_number,
std::function<void(Deletion const &)>);
// 삭제 기록
void write(uint64_t block_number,
std::vector<Deletion> const &);
};
```

동작 원리

1. 전체 동기화 흐름

```
┌──────────────────────────────────────────────────────────────────────┐
| Consensus Component |
| (monad-node) |
└──────────────────────────────────────────────────────────────────────┘
| ▲
| 1. handle_target() | 6. finalize()
| (목표 블록 헤더) | (검증 완료)
▼ |
┌──────────────────────────────────────────────────────────────────────┐
| Statesync Client Context |
| ┌──────────────────────────────────────────────────────────────┐ |
| | 2. send_request() × 256 | |
| | (각 prefix별 요청) | |
| └──────────────────────────────────────────────────────────────┘ |
| | ▲ |
| | 3. SYNC_TYPE_REQUEST | 5. DONE |
| ▼ | |
| ┌──────────────────────────────────────────────────────────────┐ |
| | 4. handle_upsert() | |
| | - UPSERT_CODE | |
| | - UPSERT_ACCOUNT | |
```

```
| | - UPSERT_STORAGE | |
| | - UPSERT_*_DELETE | |
| | - UPSERT_HEADER | |
| └──────────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────────────┘
| ▲
| Network | Network
▼ |
┌──────────────────────────────────────────────────────────────────────┐
| Statesync Server Context |
| ┌──────────────────────────────────────────────────────────┐ |
| | handle_request() | |
| | - Trie 순회 | |
| | - 삭제 전송 | |
| | - 블록 헤더 전송 | |
| └──────────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────────────┘
```

2. 클라이언트 동기화 프로세스

2.1 초기화 및 목표 설정

```
// statesync_client.cpp
void  monad_statesync_client_handle_target(
```

```
monad_statesync_client_context *ctx,
unsigned char const *data, uint64_t size) {
// 목표 블록 헤더 디코딩
byte_string_view raw{data, size};
auto res = rlp::decode_block_header(raw);
auto const &tgrt = res.value();
ctx->tgrt = tgrt;
// 이미 동기화 완료된 경우
if (tgrt.number == ctx->db.get_latest_version()) {
return;
}
// 모든 prefix에 대해 동기화 요청 전송
for (size_t i = 0; i < ctx->progress.size(); ++i) {
ctx->protocol.at(i)->send_request(ctx, i);
}
}
```

2.2 동기화 요청 생성

```
// statesync_protocol.cpp
void StatesyncProtocolV1::send_request(
monad_statesync_client_context *ctx, uint64_t prefix) const {
auto tgrt = ctx->tgrt.number;
auto const &[progress, old_target] = ctx->progress[prefix];
```

```
// 시작 블록 결정
auto from = progress == INVALID_BLOCK_NUM ? 0 : progress + 1;
// 요청 전송 (99%까지 또는 목표까지)
ctx->statesync_send_request(
ctx->sync,
monad_sync_request{
.prefix = prefix,
.prefix_bytes = 1, // 256개 분할
.target = tgrt,
.from = from,
.until = from >= (tgrt * 99 / 100) ? tgrt : tgrt * 99 / 100,
.old_target = old_target
});
}
```

Prefix 분할 전략:

- 1바이트 prefix = 256개 분할
- 각 prefix는 주소 공간의 1/256을 담당
- 병렬 다운로드 가능
- 예: prefix=0x00은 0x00로 시작하는 모든 주소

2.3 데이터 수신 및 처리

```
// statesync_protocol.cpp
```

```
bool StatesyncProtocolV1::handle_upsert(
monad_statesync_client_context *ctx,
monad_sync_type type,
unsigned char const *val, uint64_t size) const {
byte_string_view raw{val, size};
switch (type) {
case SYNC_TYPE_UPSERT_CODE:
// 코드는 불변 - 해시로 저장
ctx->code.emplace(
std::bit_cast<bytes32_t>(keccak256(raw)), raw);
break;
case SYNC_TYPE_UPSERT_ACCOUNT: {
auto [addr, acct] = decode_account_db(raw).value();
acct.incarnation = Incarnation{0, 0};
account_update(*ctx, addr, acct);
break;
}
case SYNC_TYPE_UPSERT_STORAGE: {
auto addr = unaligned_load<Address>(val);
raw.remove_prefix(sizeof(Address));
auto [k, v] = decode_storage_db(raw).value();
storage_update(*ctx, addr, k, v);
break;
```

```
}
case SYNC_TYPE_UPSERT_ACCOUNT_DELETE:
account_update(*ctx, unaligned_load<Address>(val), std::nullopt);
break;
case SYNC_TYPE_UPSERT_STORAGE_DELETE: {
auto addr = unaligned_load<Address>(val);
raw.remove_prefix(sizeof(Address));
auto key = rlp::decode_bytes32_compact(raw).value();
storage_update(*ctx, addr, key, {});
break;
}
case SYNC_TYPE_UPSERT_HEADER:
ctx->hdrs[res.value().number % 256] =
rlp::decode_block_header(raw).value();
break;
}
// 주기적 커밋 (1M upsert마다)
if ((++ctx->n_upserts % (1 << 20)) == 0) {
ctx->commit();
}
return true;
}
```

2.4 상태 업데이트 로직

```
// statesync_protocol.cpp (익명 네임스페이스)
void  account_update(
monad_statesync_client_context &ctx,
Address const  &addr,
std::optional<Account> const  &acct) {
// 코드 해시 추적
if (acct.has_value()) {
auto  const &hash = acct.value().code_hash;
if (hash != NULL_HASH) {
ctx.seen_code.emplace(hash);
}
}
auto it = ctx.deltas.find(addr);
bool updated = it != ctx.deltas.end();
// 버퍼링된 스토리지 처리 (계정 없이 스토리지만 먼저 도착)
if (ctx.buffered.contains(addr)) {
if (acct.has_value()) {
ctx.deltas.emplace(
addr,
std::make_pair(acct.value(),
std::move(ctx.buffered.at(addr))));
```

```
}
ctx.buffered.erase(addr);
}
// 새 계정
else  if (!updated) {
if (acct.has_value()) {
ctx.deltas.emplace(addr,
std::make_pair(acct.value(), StorageDeltas{}));
}
else  if (ctx.tdb.read_account(addr).has_value()) {
ctx.deltas.emplace(addr, std::nullopt); // 삭제 표시
}
}
// Incarnation 변경 (계정 재생성)
else  if (acct.has_value() && !it->second.has_value()) {
ctx.commit(); // 이전 상태 커밋
account_update(ctx, addr, acct); // 재귀 호출
}
// 기존 계정 업데이트
else  if (acct.has_value()) {
std::get<Account>(it->second.value()) = acct.value();
}
// 계정 삭제
```

```
else  if (ctx.tdb.read_account(addr).has_value()) {
it->second = std::nullopt;
}
else {
ctx.deltas.erase(it);
}
}


void  storage_update(
monad_statesync_client_context &ctx,
Address const  &addr,
bytes32_t const  &key, bytes32_t const  &val) {
auto it = ctx.deltas.find(addr);
bool updated = it != ctx.deltas.end();
// 버퍼링된 스토리지 (계정 없음)
if (ctx.buffered.contains(addr)) {
if (val == bytes32_t{}) {
ctx.buffered[addr].erase(key);
if (ctx.buffered[addr].empty()) {
ctx.buffered.erase(addr);
}
}
else {
```

```
ctx.buffered[addr][key] = val;
}
}
// 기존 계정의 스토리지
else  if (updated && it->second.has_value()) {
std::get<StorageDeltas>(it->second.value())[key] = val;
}
// Incarnation 후 스토리지
else  if (updated && !it->second.has_value() && val != bytes32_t{}) {
ctx.commit();
storage_update(ctx, addr, key, val);
}
// 새 스토리지
else {
auto orig = ctx.tdb.read_account(addr);
if (orig.has_value()) {
ctx.deltas.emplace(addr,
std::make_pair(orig.value(), StorageDeltas{{key, val}}));
}
else {
ctx.buffered.emplace(addr, StorageDeltas{{key, val}});
}
}
```

```
}
```

2.5 커밋 및 완료

```
// statesync_client_context.cpp
void monad_statesync_client_context::commit() {
std::deque<mpt::Update> alloc;
std::deque<byte_string> bytes_alloc;
std::deque<hash256> hash_alloc;
// 계정 및 스토리지 업데이트 생성
UpdateList accounts;
for (auto  const &[addr, delta] : deltas) {
UpdateList storage;
std::optional<byte_string_view> value;
if (delta.has_value()) {
auto  const &[acct, deltas] = delta.value();
value = bytes_alloc.emplace_back(encode_account_db(addr, acct));
// 스토리지 업데이트
for (auto  const &[key, val] : deltas) {
storage.push_front(alloc.emplace_back(Update{
.key = hash_alloc.emplace_back(keccak256(key.bytes)),
.value = val == bytes32_t{} ? std::nullopt
: std::make_optional(bytes_alloc.emplace_back(
encode_storage_db(key, val))),
```

```
.incarnation = false,
.next = UpdateList{},
.version = static_cast<int64_t>(current)
}));
}
}
// 계정 업데이트
accounts.push_front(alloc.emplace_back(Update{
.key = hash_alloc.emplace_back(keccak256(addr.bytes)),
.value = value,
.incarnation = false,
.next = std::move(storage),
.version = static_cast<int64_t>(current)
}));
}
// 코드 업데이트
UpdateList code_updates;
for (auto  const &[hash, bytes] : code) {
code_updates.push_front(alloc.emplace_back(Update{
.key = NibblesView{hash},
.value = bytes,
.incarnation = false,
.next = UpdateList{},
```

```
.version = static_cast<int64_t>(current)
}));
}
// Trie 구조 생성: finalized -> {state, code, block_header}
auto state_update = Update{
.key = state_nibbles,
.value = byte_string_view{},
.incarnation = false,
.next = std::move(accounts),
.version = static_cast<int64_t>(current)
};
auto code_update = Update{
.key = code_nibbles,
.value = byte_string_view{},
.incarnation = false,
.next = std::move(code_updates),
.version = static_cast<int64_t>(current)
};
auto block_header_update = Update{
.key = block_header_nibbles,
.value = rlp::encode_block_header(tgrt),
.incarnation = true,
.next = UpdateList{},
```

```
.version = static_cast<int64_t>(current)
};
UpdateList updates;
updates.push_front(state_update);
updates.push_front(code_update);
updates.push_front(block_header_update);
Update finalized{
.key = finalized_nibbles,
.value = byte_string_view{},
.incarnation = false,
.next = std::move(updates),
.version = static_cast<int64_t>(current)
};
UpdateList finalized_updates;
finalized_updates.push_front(finalized);
// 데이터베이스에 쓰기 (압축 비활성화)
db.upsert(std::move(finalized_updates), current, false, false);
tdb.set_block_and_prefix(current);
// 버퍼 정리
code.clear();
deltas.clear();
}
```

2.6 동기화 완료 처리

```
// statesync_client.cpp
void monad_statesync_client_handle_done(
monad_statesync_client_context *ctx,
monad_sync_done msg) {
auto &[progress, old_target] = ctx->progress.at(msg.prefix);
progress = msg.n;
old_target = ctx->tgrt.number;
// 아직 목표에 도달하지 못한 경우 계속 요청
if (progress != ctx->tgrt.number) {
ctx->protocol.at(msg.prefix)->send_request(ctx, msg.prefix);
}
// 모든 prefix가 완료되면 최종 커밋
if (monad_statesync_client_has_reached_target(ctx)) {
ctx->commit();
}
}

// 최종 검증 및 finalize
bool monad_statesync_client_finalize(
monad_statesync_client_context *ctx) {
auto const &tgrt = ctx->tgrt;
```

```
// 버퍼링된 스토리지가 남아있으면 실패 (계정 없는 스토리지)
if (!ctx->buffered.empty()) {
return false;
}
// 모든 코드가 수신되었는지 확인
auto latest_version = ctx->db.get_latest_version();
for_each_code(ctx->db, latest_version,
[&](bytes32_t const  &hash, byte_string_view) {
ctx->seen_code.erase(hash);
});
if (!ctx->seen_code.empty()) {
return false; // 누락된 코드 존재
}
// 버전 이동 (중간 블록 헤더 채우기)
if (latest_version != tgrt.number) {
ctx->db.move_trie_version_forward(latest_version, tgrt.number);
// 블록 헤더 체인 검증 (최대 256개)
bytes32_t expected = tgrt.parent_hash;
for (size_t i = 0; i < std::min(tgrt.number, 256ul); ++i) {
auto v = tgrt.number - i - 1;
auto  const &hdr = ctx->hdrs[v % 256];
auto rlp = rlp::encode_block_header(hdr);
auto hash = to_bytes(keccak256(rlp));
```

```
if (hash != expected) {
return false; // 헤더 체인 불일치
}
expected = hdr.parent_hash;
// 블록 헤더 저장
Update block_header_update{
.key = block_header_nibbles,
.value = rlp,
.incarnation = true,
.next = UpdateList{},
.version = static_cast<int64_t>(v)
};
// ... (finalized 구조로 감싸서 저장)
ctx->db.upsert(std::move(finalized_updates), v, false, false);
}
}
// Finalize 버전 업데이트
ctx->db.update_finalized_version(tgrt.number);
// 최종 상태 루트 검증
TrieDb db{ctx->db};
return  db.state_root() == tgrt.state_root;
}
```

3. 서버 동기화 프로세스

3.1 요청 처리

```
// statesync_server.cpp

bool  statesync_server_handle_request(

monad_statesync_server *sync,

monad_sync_request rq) {

auto *ctx = sync->context;

auto &db = *ctx->ro;

// 블록 헤더 전송 (prefix < 256인 경우)

if (rq.prefix < 256 && rq.target > rq.prefix) {

auto version = rq.target - rq.prefix - 1;

auto root = db.load_root_for_version(version);

auto res = db.find(root,

concat(FINALIZED_NIBBLE, BLOCKHEADER_NIBBLE), version);

if (res.has_value()) {

auto  const &val = res.value().node->value();

sync->statesync_server_send_upsert(

sync->net,

SYNC_TYPE_UPSERT_HEADER,

val.data(), val.size(),

nullptr, 0);

}
```

```
}

// 삭제 전송 (증분 동기화)

if (!send_deletion(sync, rq, *ctx)) {

return false;

}

// Trie 순회 및 데이터 전송

auto bytes = from_prefix(rq.prefix, rq.prefix_bytes);

auto root = db.load_root_for_version(rq.target);

auto finalized_root_res = db.find(root, finalized_nibbles, rq.target);

Traverse traverse(sync, NibblesView{bytes}, rq.from, rq.until);

if (!db.traverse(finalized_root, traverse, rq.target)) {

return false;

}

return true;

}
```

3.2 삭제 전송

```
// statesync_server.cpp

bool  send_deletion(

monad_statesync_server *sync,

monad_sync_request const  &rq,

monad_statesync_server_context &ctx) {

if (rq.old_target == INVALID_BLOCK_NUM) {
```

```
return true; // 전체 동기화 (삭제 불필요)
}
auto prefix = from_prefix(rq.prefix, rq.prefix_bytes);
// 각 블록의 삭제를 순회
for (uint64_t i = rq.old_target + 1; i <= rq.target; ++i) {
ctx.deletions.for_each(i, [&](Deletion const  &deletion) {
auto  const &[addr, key] = deletion;
auto hash = keccak256(addr.bytes);
byte_string_view view{hash.bytes, sizeof(hash.bytes)};
// Prefix 필터링
if (!view.starts_with(prefix)) {
return;
}
// 계정 삭제
if (!key.has_value()) {
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_ACCOUNT_DELETE,
reinterpret_cast<unsigned  char  const *>(&addr),
sizeof(addr),
nullptr, 0);
}
// 스토리지 삭제
```

```
else {
auto skey = rlp::encode_bytes32_compact(key.value());
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_STORAGE_DELETE,
reinterpret_cast<unsigned char const *>(&addr),
sizeof(addr),
skey.data(), skey.size());
}
});
}
return true;
}
```

3.3 Trie 순회

```
// statesync_server.cpp
struct Traverse : public TraverseMachine {
unsigned char nibble; // 현재 nibble (STATE 또는 CODE)
unsigned depth; // 현재 깊이
Address addr; // 현재 계정 주소
monad_statesync_server *sync;
NibblesView prefix; // 필터링할 prefix
```

```
uint64_t from; // 시작 블록

uint64_t until; // 종료 블록

// 노드 방문 (하향)

virtual bool down(unsigned char branch, Node const &node) override {

if (branch == INVALID_BRANCH) {

return true; // 루트

}

if (depth == 0 && nibble == INVALID_BRANCH) {

nibble = branch; // STATE 또는 CODE 선택

return true;

}

// Prefix 필터링

auto ext = node.path_nibble_view();

for (auto i = depth + 1; i < prefix.nibble_size(); ++i) {

auto j = i - (depth + 1);

if (j >= ext.nibble_size()) break;

if (ext.get(j) != prefix.get(i)) {

return false; // Prefix 불일치

}

}

// 버전 필터링

auto v = static_cast<uint64_t>(node.version);

if (v < from) {
```

```
return false; // 너무 오래된 버전
}
depth += 1 + ext.nibble_size();
constexpr  unsigned HASH_SIZE = KECCAK256_SIZE * 2; // 64 nibbles
bool account = depth == HASH_SIZE && nibble == STATE_NIBBLE;
// 계정 주소 추출
if (account && node.number_of_children() > 0) {
auto raw = node.value();
auto res = decode_account_db(raw);
addr = std::get<Address>(res.assume_value());
}
// 값이 있고 범위 내이면 전송
if (node.has_value() && v <= until) {
if (nibble == CODE_NIBBLE) {
// 코드 전송
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_CODE,
nullptr, 0,
node.value().data(), node.value().size());
}
else  if (depth == HASH_SIZE) {
// 계정 전송
```

```
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_ACCOUNT,
nullptr, 0,
node.value().data(), node.value().size());
}
else {
// 스토리지 전송 (주소 포함)
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_STORAGE,
reinterpret_cast<unsigned char *>(&addr),
sizeof(addr),
node.value().data(), node.value().size());
}
}
return true;
}
// 노드 방문 (상향)
virtual void up(unsigned char, Node const &node) override {
if (depth == 0) {
nibble = INVALID_BRANCH;
return;
```

```
}
depth -= 1 + node.path_nibbles_len();
}
// 자식 방문 여부 결정
virtual bool should_visit(
Node const &node, unsigned char branch) override {
if (depth == 0 && nibble == INVALID_BRANCH) {
return branch == STATE_NIBBLE || branch == CODE_NIBBLE;
}
// 서브트리 최소 버전 확인
auto v = node.subtrie_min_version(node.to_child_index(branch));
if (static_cast<uint64_t>(v) > until) {
return false; // 너무 새로운 버전
}
// Prefix 확인
return depth >= prefix.nibble_size() ||
prefix.get(depth) == branch;
}
};
```

3.4 삭제 추적

```
// statesync_server_context.cpp
void on_commit(
```

```
monad_statesync_server_context &ctx,
StateDeltas const  &state_deltas,
uint64_t  n, bytes32_t const  &block_id) {
auto &deletions = ctx.proposals
.emplace_back(ProposedDeletions{
.block_number = n,
.block_id = block_id,
.deletions = {}
}).deletions;
// 상태 변경사항에서 삭제 추출
for (auto  const &[addr, delta] : state_deltas) {
auto  const &account = delta.account.second;
// 스토리지 삭제
if (account.has_value()) {
for (auto  const &[key, delta] : delta.storage) {
if (delta.first != delta.second &&
delta.second == bytes32_t{}) {
deletions.emplace_back(addr, key);
}
}
}
// 계정 삭제 또는 incarnation 변경
if (delta.account.first != account) {
```

```
bool incarnation =
account.has_value() &&
delta.account.first.has_value() &&
delta.account.first->incarnation != account->incarnation;
if (incarnation || !account.has_value()) {
deletions.emplace_back(addr, std::nullopt);
}
}
}
}

void  on_finalize(
monad_statesync_server_context &ctx,
uint64_t  block_number,
bytes32_t const  &block_id) {
auto &proposals = ctx.proposals;
// 해당 블록 ID 찾기
auto it = std::find_if(proposals.begin(), proposals.end(),
[&block_id](auto  const  &p) {
return  p.block_id == block_id;
});
if (it == proposals.end()) {
return;
```

```
}
// Finalized 삭제 버퍼에 기록
ctx.deletions.write(block_number, it->deletions);
// 오래된 제안 정리
proposals.erase(
std::remove_if(proposals.begin(), proposals.end(),
[block_number](ProposedDeletions const  &p) {
return  p.block_number <= block_number;
}),
proposals.end());
}
```

3.5 삭제 버퍼 관리

```
// statesync_server_context.cpp
void FinalizedDeletions::write(
uint64_t  block_number,
std::vector<Deletion> const  &deletions) {
end_block_number_ = block_number;
// 삭제가 너무 많으면 버퍼 초기화
if (deletions.size() > MAX_DELETIONS) {
for (uint64_t i = 0; i < MAX_ENTRIES; ++i) {
clear_entry(i);
```

```
}
start_block_number_ = INVALID_BLOCK_NUM;
return;
}
if (start_block_number_ == INVALID_BLOCK_NUM) {
start_block_number_ = end_block_number_;
}
auto target_idx = end_block_number_ % MAX_ENTRIES;
clear_entry(target_idx);
// 공간 확보 (FIFO)
while ((free_end_ - free_start_) < deletions.size()) {
clear_entry(start_block_number_ % MAX_ENTRIES);
}
set_entry(target_idx, end_block_number_, deletions);
}


void FinalizedDeletions::set_entry(
uint64_t  i, uint64_t  block_number,
std::vector<Deletion> const  &deletions) {
auto &entry = entries_[i];
std::lock_guard lock{entry.mutex};
entry.block_number = block_number;
entry.idx = free_start_;
```

```
entry.size = deletions.size();

// 링 버퍼에 삭제 복사

for (auto  const &deletion : deletions) {

deletions_[free_start_++ % MAX_DELETIONS] = deletion;

}

}
```

최적화 기법

1. 병렬 다운로드

Prefix 기반 분할

```
주소 공간 분할 (1바이트 prefix = 256개 분할):


Prefix 0x00: 0x00000000... ~ 0x00FFFFFF...

Prefix 0x01: 0x01000000... ~ 0x01FFFFFF...

...

Prefix 0xFF: 0xFF000000... ~ 0xFFFFFFFF...


각 prefix는 독립적으로 동기화 가능

→ 최대 256개 병렬 다운로드
```

장점:

- 네트워크 대역폭 최대 활용

- 여러 피어로부터 동시 다운로드
- 부분 실패 시 해당 prefix만 재시도

구현:

```
// 각 prefix별 진행 상황 추적
std::vector<std::pair<uint64_t, uint64_t>> progress; // [진행, 이전_목표]


// 모든 prefix 동시 요청
for (size_t i = 0; i < 256; ++i) {
ctx->protocol.at(i)->send_request(ctx, i);
}


// 각 prefix 독립적으로 완료 처리
void  handle_done(monad_sync_done msg) {
auto &[progress, old_target] = ctx->progress.at(msg.prefix);
progress = msg.n;
if (progress != ctx->tgrt.number) {
ctx->protocol.at(msg.prefix)->send_request(ctx, msg.prefix);
}
}
```

2. 증분 동기화

변경사항만 전송

```
struct monad_sync_request {
uint64_t target; // 목표 블록
uint64_t from; // 시작 블록
uint64_t until; // 종료 블록
uint64_t old_target; // 이전 목표 (증분 동기화용)
};
```

전체 동기화 vs 증분 동기화:

```
전체 동기화 (old_target = INVALID):
- 모든 계정 및 스토리지 전송
- 삭제 전송 불필요

증분 동기화 (old_target 지정):
- old_target+1 ~ target 범위의 변경사항만 전송
- 삭제 명시적 전송 (UPSERT_*_DELETE)
- 네트워크 효율성 극대화
```

삭제 추적:

```
// 커밋 시 삭제 기록
void on_commit(StateDeltas const &state_deltas, uint64_t n) {
for (auto const &[addr, delta] : state_deltas) {
// 스토리지 삭제
```

```
for (auto  const &[key, delta] : delta.storage) {
if (delta.second == bytes32_t{}) {
deletions.emplace_back(addr, key);
}
}
// 계정 삭제
if (!delta.account.second.has_value()) {
deletions.emplace_back(addr, std::nullopt);
}
}
}

// 동기화 시 삭제 전송
for (uint64_t i = old_target + 1; i <= target; ++i) {
ctx.deletions.for_each(i, [&](Deletion const  &deletion) {
send_delete(deletion);
});
}
```

3. 버전 필터링

Trie 순회 최적화

```
// 노드 버전 확인
```

```
auto v = static_cast<uint64_t>(node.version);
if (v < from || v > until) {
return false; // 범위 밖 노드 스킵
}

// 서브트리 최소 버전 확인
auto min_v = node.subtrie_min_version(child_index);
if (static_cast<uint64_t>(min_v) > until) {
return false; // 전체 서브트리 스킵
}
```

효과:

- 불필요한 노드 방문 최소화
- 디스크 I/O 감소
- 순회 속도 향상

4. 주기적 커밋

```
// 1M upsert마다 커밋
if ((++ctx->n_upserts % (1 << 20)) == 0) {
ctx->commit();
}
```

장점:

- 메모리 사용량 제한
- 중단 시 재시작 지점 제공
- 진행 상황 추적 가능

단점:

- 커밋 오버헤드
- 디스크 I/O 증가

트레이드오프:

- 1M = 약 100MB 메모리 사용 (추정)
- 커밋 빈도 ↑ → 메모리 ↓, I/O ↑
- 커밋 빈도 ↓ → 메모리 ↑, I/O ↓

5. 압축 비활성화

```
// 동기화 중에는 압축 비활성화
db.upsert(std::move(finalized_updates), current,
false, // enable_compaction = false
false); // rewind = false
```

이유:

- 동기화 중 압축은 불필요 (최종 상태만 필요)
- 압축 오버헤드 제거
- 쓰기 속도 향상

동기화 완료 후:

- 압축 활성화

- 오래된 버전 정리

6. io_uring 활용

```
// 클라이언트 초기화 시 io_uring 설정
monad::mpt::OnDiskDbConfig{
.append = true,
.compaction = false,
.rewind_to_latest_finalized = true,
.rd_buffers = 8192, // 읽기 버퍼 (동기화 중 많은 읽기)
.wr_buffers = 32, // 쓰기 버퍼
.uring_entries = 128, // io_uring 큐 크기
.sq_thread_cpu = sq_thread_cpu, // 전용 CPU (옵션)
.dbname_paths = dbname_paths
}
```

최적화:

- 비동기 디스크 I/O
- 배치 읽기/쓰기
- CPU 오버헤드 최소화

7. 검증 전략

단계별 검증

```
bool  monad_statesync_client_finalize(
```

```
monad_statesync_client_context *ctx) {

// 1. 버퍼링된 스토리지 확인

if (!ctx->buffered.empty()) {

return false; // 계정 없는 스토리지 존재

}

// 2. 코드 완전성 확인

for_each_code(ctx->db, latest_version,

[&](bytes32_t const  &hash, byte_string_view) {

ctx->seen_code.erase(hash);

});

if (!ctx->seen_code.empty()) {

return false; // 누락된 코드

}

// 3. 블록 헤더 체인 검증

bytes32_t expected = tgrt.parent_hash;

for (size_t i = 0; i < std::min(tgrt.number, 256ul); ++i) {

auto v = tgrt.number - i - 1;

auto  const &hdr = ctx->hdrs[v % 256];

auto hash = to_bytes(keccak256(rlp::encode_block_header(hdr)));

if (hash != expected) {

return false; // 헤더 체인 불일치

}

expected = hdr.parent_hash;
```

```
}

// 4. 최종 상태 루트 검증

TrieDb db{ctx->db};

return  db.state_root() == tgrt.state_root;

}
```

검증 레벨:

1. **구조적 검증**: 데이터 무결성 (버퍼, 코드)
2. **체인 검증**: 블록 헤더 연결성
3. **상태 검증**: Merkle 루트 일치

8. 네트워크 효율성

메시지 크기 최적화

```
// RLP 압축 인코딩

auto skey = rlp::encode_bytes32_compact(key.value());


// 불필요한 데이터 제거

if (val == bytes32_t{}) {

// 0 값은 삭제로 처리 (전송 안 함)

storage_update(ctx, addr, key, {});

}
```

배치 전송

```
단일 메시지:
- 타입 (1바이트)
- 크기 (가변)
- 데이터

배치 전송:
- 여러 upsert를 연속으로 전송
- TCP 버퍼링 활용
- 네트워크 왕복 최소화
```

압축 (미래 개선)

현재는 압축 미사용, 향후 추가 가능:

- LZ4/Zstd 압축
- 델타 인코딩
- Bloom 필터 (중복 방지)

extern "C" API 인터페이스

클라이언트 API

```
// 컨텍스트 생성
struct monad_statesync_client_context *
monad_statesync_client_context_create(
```

```
char const *const *dbname_paths, // 데이터베이스 경로 배열
size_t len, // 경로 개수
unsigned sq_thread_cpu, // io_uring 전용 CPU
struct monad_statesync_client *, // 클라이언트 핸들
void (*statesync_send_request)( // 요청 전송 콜백
struct monad_statesync_client *,
struct monad_sync_request));

// Prefix 설정
uint8_t  monad_statesync_client_prefix_bytes(); // 1바이트
size_t  monad_statesync_client_prefixes(); // 256개

// 동기화 상태 확인
bool  monad_statesync_client_has_reached_target(
struct monad_statesync_client_context const  *);

// 이벤트 핸들러
void  monad_statesync_client_handle_new_peer(
struct monad_statesync_client_context *,
uint64_t  prefix, // Peer가 담당하는 prefix
uint32_t  version); // 프로토콜 버전

void  monad_statesync_client_handle_target(
```

```
struct monad_statesync_client_context *,
unsigned char const *, // 목표 블록 헤더 (RLP)
uint64_t); // 크기

bool monad_statesync_client_handle_upsert(
struct monad_statesync_client_context *,
uint64_t prefix, // 데이터 소스 prefix
enum monad_sync_type, // 메시지 타입
unsigned char const *, // 데이터
uint64_t); // 크기

void monad_statesync_client_handle_done(
struct monad_statesync_client_context *,
struct monad_sync_done); // 완료 메시지

// 최종 검증 및 finalize
bool monad_statesync_client_finalize(
struct monad_statesync_client_context *);

// 컨텍스트 파괴
void monad_statesync_client_context_destroy(
struct monad_statesync_client_context *);
```

서버 API

```
// 서버 생성
struct monad_statesync_server *monad_statesync_server_create(
struct monad_statesync_server_context *, // 서버 컨텍스트
struct monad_statesync_server_network *, // 네트워크 핸들
ssize_t (*statesync_server_recv)( // 수신 콜백
struct monad_statesync_server_network *,
unsigned char *, size_t),
void (*statesync_server_send_upsert)( // upsert 전송 콜백
struct monad_statesync_server_network *,
enum monad_sync_type,
unsigned char const *v1, uint64_t size1,
unsigned char const *v2, uint64_t size2),
void (*statesync_server_send_done)( // 완료 전송 콜백
monad_statesync_server_network *,
struct monad_sync_done));

// 요청 처리 (블로킹)
void monad_statesync_server_run_once(
struct monad_statesync_server *);

// 서버 파괴
```

```
void monad_statesync_server_destroy(
struct monad_statesync_server *);
```

버전 호환성 API

```
// 현재 프로토콜 버전
uint32_t monad_statesync_version();

// 클라이언트 호환성 확인
bool monad_statesync_client_compatible(uint32_t version);
```

합의 컴포넌트와의 통신

인터페이스 설계

```
┌──────────────────────────────────────────────────────────────┐
| Consensus Component (Go/Rust) |
| |
| ┌──────────────────────────────────────────────────┐ |
| | Statesync Client Wrapper | |
| | - monad_statesync_client_context_create() | |
| | - monad_statesync_client_handle_*() | |
| └──────────────────────────────────────────────────┘ |
| | ▲ |
```

```
| | C Function Calls | |
| ▼ | |
| ┌──────────────────────────────────────────────────────┐ |
| | Callback Functions (Go/Rust → C++) | |
| | - statesync_send_request() | |
| └──────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────┘
| ▲
| Network (P2P) |
▼ |
┌──────────────────────────────────────────────────────────────┐
| Execution Component (C++) |
| |
| ┌──────────────────────────────────────────────────────┐ |
| | libmonad_execution.so | |
| | - monad_statesync_client_context (C++ 구현) | |
| └──────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────────┘
```

콜백 구조

클라이언트 → 합의 (요청 전송)

```
// C++ 측 (실행 컴포넌트)
```

```
void StatesyncProtocolV1::send_request(
monad_statesync_client_context *ctx, uint64_t  prefix) const {
// 요청 메시지 생성
monad_sync_request req{
.prefix = prefix,
.prefix_bytes = 1,
.target = ctx->tgrt.number,
.from = progress + 1,
.until = target * 99 / 100,
.old_target = old_target
};
// 합의 컴포넌트 콜백 호출
ctx->statesync_send_request(ctx->sync, req);
}


// Go/Rust 측 (합의 컴포넌트)
void  statesync_send_request_callback(
monad_statesync_client *client,
monad_sync_request req) {
// P2P 네트워크로 요청 전송
peer := selectPeerForPrefix(req.prefix)
peer.Send(SYNC_TYPE_REQUEST, &req)
```

```
}
```

합의 → 클라이언트 (데이터 수신)

```
// Go/Rust 측 (합의 컴포넌트)
func handleSyncMessage(msg *SyncMessage) {
switch msg.Type {
case SYNC_TYPE_TARGET:
monad_statesync_client_handle_target(
client_ctx,
msg.Data,
uint64(len(msg.Data)))
case SYNC_TYPE_UPSERT_ACCOUNT:
monad_statesync_client_handle_upsert(
client_ctx,
msg.Prefix,
SYNC_TYPE_UPSERT_ACCOUNT,
msg.Data,
uint64(len(msg.Data)))
case SYNC_TYPE_DONE:
done := (*monad_sync_done)(unsafe.Pointer(&msg.Data[0]))
monad_statesync_client_handle_done(client_ctx, *done)
}
```

```
}
```

서버 측 콜백

```
// C++ 측 (실행 컴포넌트)
void monad_statesync_server_run_once(
monad_statesync_server *sync) {
unsigned char buf[sizeof(monad_sync_request)];
// 합의 컴포넌트로부터 수신
if (sync->statesync_server_recv(sync->net, buf, 1) != 1) {
return;
}
// 요청 처리
auto const &rq = unaligned_load<monad_sync_request>(buf);
monad_statesync_server_handle_request(sync, rq);
// 데이터 전송 (콜백 사용)
sync->statesync_server_send_upsert(
sync->net,
SYNC_TYPE_UPSERT_ACCOUNT,
data, size,
nullptr, 0);
// 완료 전송
sync->statesync_server_send_done(
```

```
sync->net,

monad_sync_done{.success = true, .prefix = rq.prefix, .n = rq.until});

}
```

성능 특성

동기화 속도

영향 요인:

1. **네트워크 대역폭**

- 병렬 다운로드 (최대 256개 prefix)
- 피어 수
- 네트워크 지연

2. **디스크 I/O**

- io_uring 비동기 I/O
- NVMe 성능
- 쓰기 버퍼 크기

3. **상태 크기**

- 계정 수
- 스토리지 항목 수
- 코드 크기

예상 성능:

가정 :

```
- 네트워크: 1 Gbps

- 디스크: NVMe (3 GB/s 쓰기)

- 상태 크기: 100 GB

- 병렬도: 256 prefix


이론적 최대:

- 네트워크 제한: 100 GB / (1 Gbps / 8) = 800초 ≈ 13분

- 디스크 제한: 100 GB / 3 GB/s = 33초


실제 (오버헤드 포함):

- 약 20-30분 (네트워크 제한)
```

메모리 사용량

```
// 클라이언트 메모리 사용

struct monad_statesync_client_context {

// 고정 크기

std::array<BlockHeader, 256> hdrs; // ~50 KB

std::vector<std::pair<uint64_t, uint64_t>> progress; // ~4 KB

// 가변 크기 (커밋 주기에 따라)

Map<Address, StorageDeltas> buffered; // ~10-100 MB

Map<bytes32_t, byte_string> code; // ~10-50 MB

Map<Address, std::optional<StateDelta>> deltas; // ~50-200 MB
```

```
// 총: ~100-400 MB (1M upsert 기준)
};

// 서버 메모리 사용
struct monad_statesync_server_context {
// 삭제 버퍼
FinalizedDeletions deletions; // ~109 MB (고정)
// 제안된 삭제
std::deque<ProposedDeletions> proposals; // ~1-10 MB
// 총: ~110-120 MB
};
```

디스크 사용량

```
동기화 중:
- 압축 비활성화 → 더 많은 디스크 사용
- 중간 버전 저장 → 추가 공간 필요

동기화 완료 후:
- 압축 활성화 → 공간 회수
- 최종 상태만 유지

예상 디스크 사용:
```

- 최종 상태: 100 GB

- 동기화 중: 120-150 GB (20-50% 오버헤드)

- 압축 후: 100 GB

에러 처리 및 복구

클라이언트 에러 처리

```
// 1. 데이터 검증 실패
bool StatesyncProtocolV1::handle_upsert(...) {
auto res = decode_account_db(raw);
if (res.has_error()) {
return false; // 디코딩 실패 → 재요청
}
return true;
}

// 2. 최종 검증 실패
bool  monad_statesync_client_finalize(...) {
// 버퍼링된 스토리지 확인
if (!ctx->buffered.empty()) {
return false; // 계정 없는 스토리지
}
```

```
// 코드 완전성 확인
if (!ctx->seen_code.empty()) {
return false; // 누락된 코드
}
// 상태 루트 검증
if (db.state_root() != tgrt.state_root) {
return false; // 상태 불일치
}
return true;
}


// 3. 재시작 지원
monad_statesync_client_context::monad_statesync_client_context(...) {
// 최신 finalized 버전에서 재시작
.rewind_to_latest_finalized = true,
// 진행 상황 초기화
progress(prefixes, {db.get_latest_version(), db.get_latest_version()})
}
```

서버 에러 처리

```
// 1. 요청 처리 실패
bool  statesync_server_handle_request(...) {
```

```
// 루트 로드 실패
auto root = db.load_root_for_version(rq.target);
if (!root.is_valid()) {
return false;
}
// Trie 순회 실패
if (!db.traverse(finalized_root, traverse, rq.target)) {
return false;
}
return true;
}


// 2. 완료 메시지 전송
void  monad_statesync_server_handle_request(...) {
auto success = statesync_server_handle_request(sync, rq);
sync->statesync_server_send_done(
sync->net,
monad_sync_done{
.success = success,
.prefix = rq.prefix,
.n = rq.until
});
```

```
}
```

복구 전략

부분 실패:

```
Prefix 0x00: 완료
Prefix 0x01: 실패 ← 재요청
Prefix 0x02: 완료
...

→ 실패한 prefix만 재동기화
→ 다른 prefix는 영향 없음
```

전체 실패:

```
1. 데이터베이스 상태 확인
2. 최신 finalized 버전 확인
3. 해당 버전부터 재시작
4. 진행 상황 복원
```

네트워크 단절:

```
1. 주기적 커밋으로 진행 상황 저장
2. 재연결 시 마지막 커밋부터 재개
```

3. 타임아웃 후 다른 피어 시도

코드 예시

전체 동기화 흐름 예시

```
// 1. 클라이언트 초기화
auto *client_ctx = monad_statesync_client_context_create(
dbname_paths, len,
MONAD_SQPOLL_DISABLED, // 또는 CPU 번호
client_handle,
send_request_callback);

// 2. 피어 연결
for (auto  const &peer : peers) {
monad_statesync_client_handle_new_peer(
client_ctx,
peer.prefix,
peer.protocol_version);
}

// 3. 목표 블록 설정
auto target_header_rlp = get_target_header_from_consensus();
```

```
monad_statesync_client_handle_target(
client_ctx,
target_header_rlp.data(),
target_header_rlp.size());


// 4. 데이터 수신 루프
while (!monad_statesync_client_has_reached_target(client_ctx)) {
auto msg = receive_sync_message();
switch (msg.type) {
case SYNC_TYPE_UPSERT_ACCOUNT:
case SYNC_TYPE_UPSERT_STORAGE:
case SYNC_TYPE_UPSERT_CODE:
case SYNC_TYPE_UPSERT_ACCOUNT_DELETE:
case SYNC_TYPE_UPSERT_STORAGE_DELETE:
case SYNC_TYPE_UPSERT_HEADER:
if (!monad_statesync_client_handle_upsert(
client_ctx, msg.prefix, msg.type,
msg.data, msg.size)) {
// 에러 처리
request_resend(msg.prefix);
}
break;
case SYNC_TYPE_DONE:
```

```
monad_statesync_client_handle_done(
client_ctx,
*reinterpret_cast<monad_sync_done*>(msg.data));
break;
}
}

// 5. 최종 검증
if (monad_statesync_client_finalize(client_ctx)) {
LOG_INFO("State sync completed successfully");
} else {
LOG_ERROR("State sync verification failed");
// 재시도 또는 에러 처리
}

// 6. 정리
monad_statesync_client_context_destroy(client_ctx);
```

서버 예시

```
// 1. 서버 컨텍스트 생성
auto *server_ctx = new monad_statesync_server_context(trie_db);
```

```
// 2. 읽기 전용 데이터베이스 설정
server_ctx->ro = &readonly_db;


// 3. 서버 생성
auto *server = monad_statesync_server_create(
server_ctx,
network_handle,
recv_callback,
send_upsert_callback,
send_done_callback);


// 4. 요청 처리 루프
while (running) {
monad_statesync_server_run_once(server);
}


// 5. 정리
monad_statesync_server_destroy(server);
delete server_ctx;
```

프로토콜 버전 관리

버전 호환성

```
// statesync_version.cpp
uint32_t monad_statesync_version() {
return 1; // 현재 프로토콜 버전
}

bool monad_statesync_client_compatible(uint32_t version) {
return version == 1; // 버전 1만 지원
}
```

프로토콜 추상화

```
// statesync_protocol.hpp
struct StatesyncProtocol {
virtual ~StatesyncProtocol() = default;
virtual void send_request(
monad_statesync_client_context *, uint64_t prefix) const = 0;
virtual bool handle_upsert(
monad_statesync_client_context *, monad_sync_type,
unsigned char const *, uint64_t) const = 0;
};

// 버전 1 구현
struct StatesyncProtocolV1 : StatesyncProtocol {
```

```
virtual void send_request(...) const override;
virtual bool handle_upsert(...) const override;
};

// 미래 버전 (예시)
struct StatesyncProtocolV2 : StatesyncProtocol {
// 압축 지원
// 델타 인코딩
// Bloom 필터
};
```

버전 협상

```
// 클라이언트 측
void monad_statesync_client_handle_new_peer(
monad_statesync_client_context *ctx,
uint64_t prefix, uint32_t version) {
// 호환성 확인
MONAD_ASSERT(monad_statesync_client_compatible(version));
auto &ptr = ctx->protocol.at(prefix);
// 버전별 프로토콜 생성
switch (version) {
case 1:
```

```
ptr = std::make_unique<StatesyncProtocolV1>();

break;

case  2:

ptr = std::make_unique<StatesyncProtocolV2>();

break;

default:

MONAD_ASSERT(false); // 지원하지 않는 버전

}

}
```

참고 자료

관련 문서

- [01-아키텍처-개요.md](): 전체 시스템 구조
- [04-TrieDB-상태-관리.md](): Merkle Patricia Trie 구조
- [05-비동기-IO-최적화.md](): io_uring 통합

주요 파일

- `category/statesync/statesync_client.h` : 클라이언트 C API
- `category/statesync/statesync_server.h` : 서버 C API
- `category/statesync/statesync_messages.h` : 메시지 타입 정의
- `category/statesync/statesync_protocol.hpp` : 프로토콜 인터페이스
- `category/statesync/statesync_client_context.hpp` : 클라이언트 컨텍스트

- `category/statesync/statesync_server_context.hpp` : 서버 컨텍스트

핵심 개념

- **Prefix 분할**: 주소 공간을 256개로 분할하여 병렬 동기화
- **증분 동기화**: 변경사항만 전송하여 네트워크 효율성 향상
- **삭제 추적**: 명시적 삭제 메시지로 상태 일관성 보장
- **extern "C" API**: 합의 컴포넌트(Go/Rust)와 실행 컴포넌트(C++) 간 인터페이스
- **검증 가능성**: Merkle 루트를 통한 최종 상태 검증